CORE JAVA SERIES — DAY 12

Enums & Interfaces in Depth

Why enums replaced int and String constants, what the compiler turns an enum into, and its built-in methods — then interfaces end to end: contracts, constants, multiple inheritance, the diamond problem, default, static and private methods, and how an interface differs from an abstract class.

The Problem Enums Solve

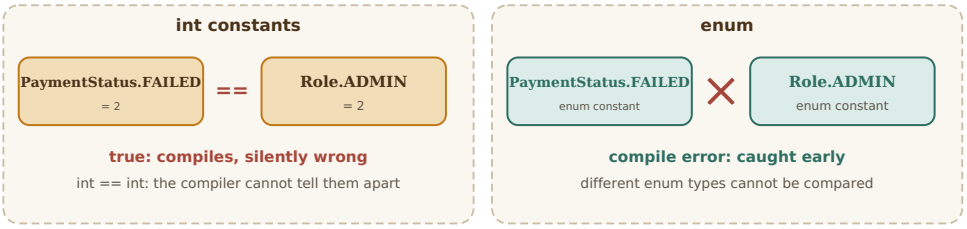
Suppose an application records a payment's status. It can only ever be **SUCCESS**, **FAILED** or **PENDING** — a fixed set of constant values that never change. How do we represent that in code? Before enums existed (before Java 5) the standard answer was a class full of `final` constants:

```
class PaymentStatus {
    public static final int SUCCESS = 1;
    public static final int FAILED  = 2;
    public static final int PENDING = 3;
}

int status = PaymentStatus.SUCCESS;
System.out.println(status);          // 1
```

`final` makes each one a constant (named in UPPER_CASE). `static` makes it belong to the class, so there is one shared copy and no object is needed — you simply write `PaymentStatus.SUCCESS`. This works, but it hides several problems:

Problem	What goes wrong
No type safety	`int status = 100;` compiles, though 100 is not a valid status. The bug appears at run time in a live application instead of at compile time.
Poor readability	`if (status == 2)` — what is 2? A reader has to open `PaymentStatus` to find out it means FAILED.
No grouping of related things	A second class `Role` has `ADMIN = 2`. Then `status == Role.ADMIN` compiles and is **true** for a FAILED payment, because both are plain ints.
Duplicates allowed	Adding `MANAGER = 2` to `Role` raises no error, so `ADMIN` and `MANAGER` become indistinguishable.



With plain ints a FAILED payment equals an ADMIN role; with enums that comparison does not even compile.

Switching to Strings does not rescue this: `"success"` and `"SUCCESS"` are different values (case sensitivity), comparing Strings is slower because it checks character by character, there is still no type safety, and duplicates are still allowed.

Enums: the Solution (Java 5)

An **enum** (enumeration, also called an enumerated type) is a **predefined set of constants**. The syntax replaces `class` with `enum` and lists the constants separated by commas:

```
enum PaymentStatus { SUCCESS, FAILED, PENDING }

PaymentStatus status = PaymentStatus.SUCCESS;     // the variable has the enum TYPE
System.out.println(status.name());                // SUCCESS

// PaymentStatus s = 100;              // compile error: cannot convert int to PaymentStatus
// PaymentStatus s = "SUCCESS";        // compile error: String is not a PaymentStatus
// if (status == "SUCCESS") ...        // compile error: incomparable types
```

The variable now has the *type* `PaymentStatus`, so it can hold only `SUCCESS`, `FAILED` or `PENDING`. Every mistake from the table above becomes a compile-time error, which is exactly where we want to find mistakes:

- **Type safety** — no random 100 and no stray String can slip in.
- **Readability** — `if (status == PaymentStatus.FAILED)` says what it means.
- **Grouping** — `PaymentStatus` and `Role` are different types and cannot be mixed up.
- **No duplicates** — each constant is its own distinct object.

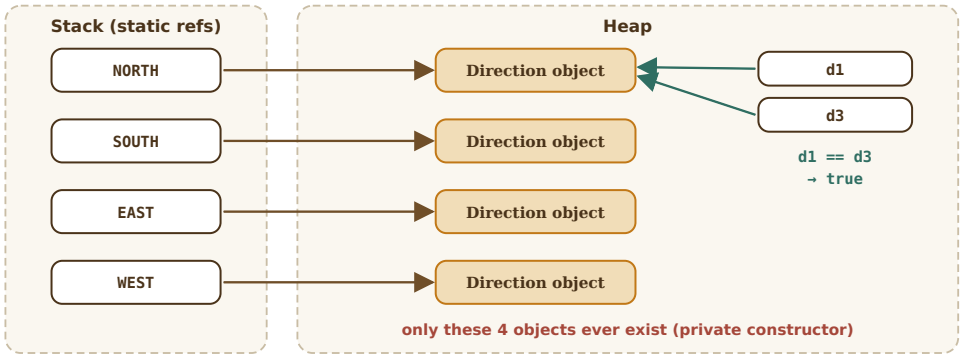
What an Enum Really Is

An enum is **nothing but a special kind of class**. When you compile, Java rewrites it as a class. Take `enum Direction { NORTH, SOUTH, EAST, WEST };` conceptually the compiler produces:

```
final class Direction extends Enum<Direction> {       // every enum extends java.lang.Enum
    public static final Direction NORTH = new Direction();
    public static final Direction SOUTH = new Direction();
    public static final Direction EAST  = new Direction();
    public static final Direction WEST  = new Direction();

    private Direction() { }                           // private constructor
}
```

- Every enum extends `java.lang.Enum` (in the always-imported `java.lang` package). The class is implicitly final, so it cannot be subclassed.
- Each constant is a `public static final` **object of the enum's own type** — not an int. You never write those keywords.
- The constructor is always **private**, so no code outside can write `new Direction()`. Exactly four objects exist for the whole program, and every reference points to one of them.



Direction.NORTH is one object; any variable assigned it points to that same object.

Since `d1` and `d3` both hold `Direction.NORTH`, they point to the same object, so `d1 == d3` is `true` — and that is why enums are compared with `==`. Here the objects hold no data of their own; they act purely as **markers** that tell you which direction (or status, or day) something is. This suits anything that is a limited set of constants: days of the week, payment or order status, log levels such as INFO / DEBUG / ERROR.

Active Recall — Part 1

Q1. Name three problems with int constants.
A. No type safety, poor readability, no grouping of related entities (plus duplicates allowed).

Q2. Why are Strings a poor substitute?
A. Case-sensitive, slower to compare, no type safety, duplicates still possible.

Q3. What does an enum compile into?
A. A class that extends java.lang.Enum, with each constant a static final object of that class and a private constructor.

Q4. Why is the enum constructor private?
A. So no outside code can create more objects; only the declared constants exist.

Q5. Why can enum constants be compared with ==?
A. Each constant is a single object, so equal constants are the very same reference.

Fields, Constructors and Methods in an Enum

Because an enum is a class, it can also carry **state** and **behaviour**. Say each direction should remember its compass degree (north 0°, east 90°, south 180°, west 270°). Add a field and a constructor, and pass the value next to each constant:

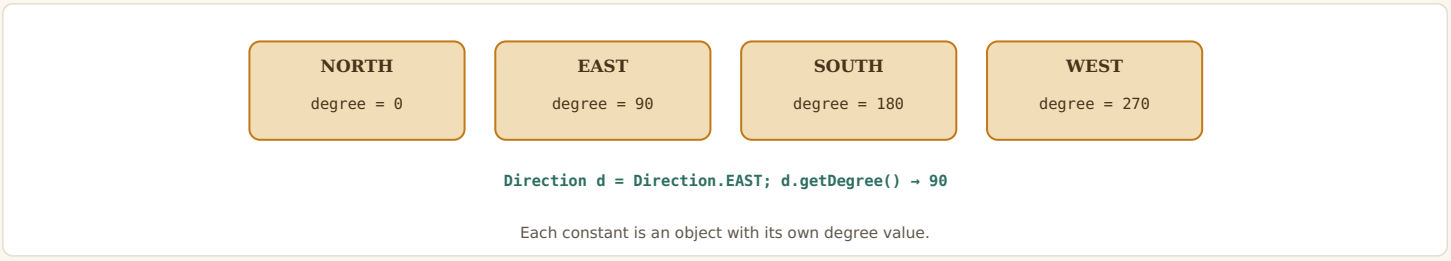
```
enum Direction {
    NORTH(0), EAST(90), SOUTH(180), WEST(270);      // semicolon: members follow

    private final int degree;                        // a normal, non-static field

    Direction(int degree) { this.degree = degree; }  // always private, even if not written
    public int getDegree() { return degree; }
}

Direction d = Direction.EAST;
System.out.println(d.getDegree());                   // 90
```

Behind the scenes `NORTH(0)` becomes `public static final Direction NORTH = new Direction(0);`. The `degree` field is not static, so it lives inside each object — the four constants are no longer empty, each one stores its own degree. Even if you write the constructor yourself, the compiler still makes it **private**, because the whole point is that no more than these objects may ever exist.



Each constant is an object with its own degree value.

A different method body per constant

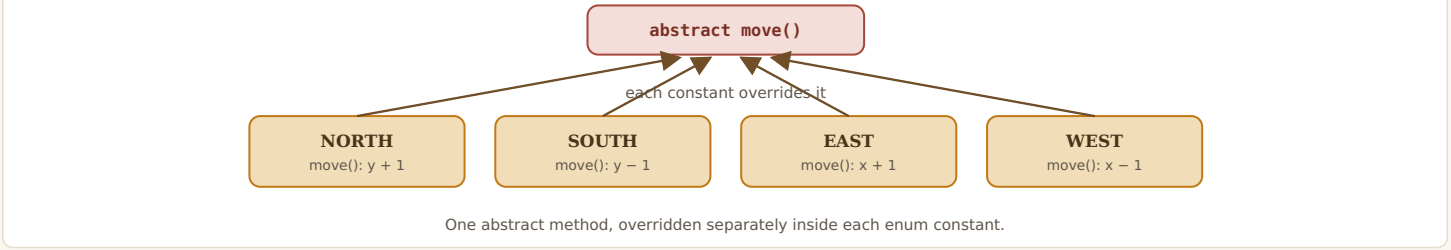
Suppose moving north should do `y + 1`, south `y − 1`, east `x + 1` and west `x − 1`: one method name, four behaviours — polymorphism. Declare the method `abstract` in the enum and give every constant its own body in braces (an anonymous class, since a constant is an object):

```
enum Direction {
    NORTH { @Override public void move() { System.out.println("y + 1"); } },
    SOUTH { @Override public void move() { System.out.println("y - 1"); } },
    EAST  { @Override public void move() { System.out.println("x + 1"); } },
    WEST  { @Override public void move() { System.out.println("x - 1"); } };

    public abstract void move();                    // every constant must define it
}

Direction d = Direction.NORTH;
d.move();                                           // y + 1
```

If a constant skips the body, the compiler complains: *"The enum constant NORTH must implement the abstract method move()"* — the same rule as any abstract method.



One abstract method, overridden separately inside each enum constant.

Built-in Enum Methods

Every enum extends `java.lang.Enum` and so inherits some ready-made methods, and the compiler adds two more. This split is a favourite interview question: **only `name()` and `ordinal()` come from the `Enum` class; `values()` and `valueOf(String)` are generated by the compiler.**

Method	Comes from	What it does
`values()`	Compiler-generated, static	Returns an array of all the constants, so you can iterate over the enum.
`valueOf(String)`	Compiler-generated, static	Converts a String into the matching constant. Case-sensitive; no match throws `IllegalArgumentException`.
`name()`	`Enum` class (final)	The constant's exact declared name, e.g. "EAST".
`ordinal()`	`Enum` class	Zero-based position in the declaration order (NORTH = 0, SOUTH = 1, ...).

```
enum Direction { NORTH, SOUTH, EAST, WEST }

for (Direction d : Direction.values())          // enhanced for loop: visits each element
    System.out.println(d.name());               // NORTH SOUTH EAST WEST

Direction d = Direction.valueOf("EAST");
System.out.println(d.name());                   // EAST
System.out.println(d.ordinal());                // 2
Direction.valueOf("East");                      // IllegalArgumentException: wrong case
```

The **enhanced for loop** `for (Type x : array)` walks straight over the elements instead of over indexes. `values()` is what makes it possible to loop over every constant — for example to fill a drop-down menu from an enum. `valueOf` is for when you receive a String and need the matching constant.

name() vs toString()

Both give the constant's name by default (and `println(d)` calls `toString()` automatically). The difference: `toString()` **can be overridden**, `name()` is **final** and cannot. Use `name()` when you need the real name every time, `toString()` when you want a custom display text.

```
enum Direction {
    NORTH, SOUTH, EAST, WEST;

    @Override
    public String toString() { return name() + " Direction"; }
    // overriding name() instead -> error: cannot override the final method from Enum
}

Direction d = Direction.EAST;
System.out.println(d);          // EAST Direction   (toString)
System.out.println(d.name());   // EAST             (unchanged)
```

How values() and valueOf() work inside

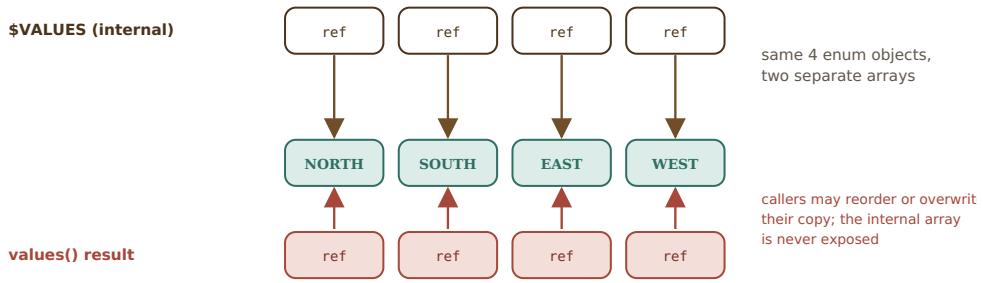
Why does the compiler generate these two? Because every enum has different constants, and the shared `Enum` base class cannot know them in advance. So for each enum the compiler writes an internal array of all its constants plus the two methods:

```
private static final Direction[] $VALUES = { NORTH, SOUTH, EAST, WEST };

public static Direction[] values() { return $VALUES.clone(); }

public static Direction valueOf(String s) {
    return (Direction) Enum.valueOf(Direction.class, s);   // delegates to Enum's own version
}
```

- `values()` returns a **clone** of the array, never the array itself. Otherwise a caller could overwrite its elements and damage the enum for the whole program. The clone holds the *same four objects* — correct, because there must only ever be one object per constant.
- `Enum` also has a `valueOf`, but it takes two parameters (the enum's class and the String). The generated one-parameter `valueOf(String)` simply calls it with `Direction.class` filled in. (`.class` belongs to reflection, covered later.)



values() hands out a fresh array pointing at the same enum objects.

Rules and uses

- An enum **cannot extend another class** — it already extends `Enum`, and Java allows only one parent class. It **can implement any number of interfaces**.
- Constructors, fields, getters and normal methods all work as in any class; the inherited `toString()` is the one you would typically override.
- Use enums for anything expressible as a limited set of constants: days of the week, payment or order status, log levels.

Active Recall — Part 2

Q1. How do you give each enum constant its own data?

A. Add a non-static field and a constructor, and pass values next to each constant, e.g. NORTH(0).

Q2. How can each constant behave differently?

A. Declare an abstract method in the enum and give every constant its own body.

Q3. Which built-in methods come from Enum and which from the compiler?

A. name() and ordinal() from Enum; values() and valueOf(String) are compiler-generated.

Q4. What happens with valueOf("East") for a constant EAST?

A. IllegalArgumentException — matching is case-sensitive.

Q5. name() vs toString()? And why does values() return a clone?

A. toString() can be overridden, name() is final. The clone protects the internal array from modification.

Q6. Can an enum extend a class? Implement interfaces?

A. No (it extends Enum already); yes, any number of interfaces.

Interfaces: the Contract

Java has two ways to implement abstraction: abstract classes and interfaces. An **interface** declares what something can do without saying how. A class is a blueprint of an *object*; an interface is a **blueprint of behaviour** — you can never create an object of an interface. That is why an interface is also called a **contract**: any class that implements it promises to provide a definition for every one of its methods.

```
interface Car {
    void drive();                           // declared only, no body
}

class Thar implements Car {
    @Override
    public void drive() {                   // must be public (see below)
        System.out.println("Thar is driving");
    }
}

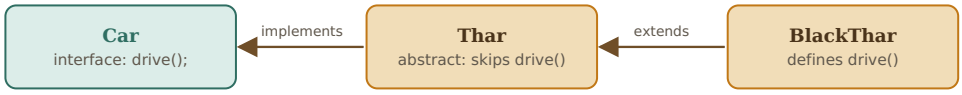
Car c = new Thar();                         // Car is the type, Thar is the object
c.drive();                                  // Thar is driving
```

- **Methods in an interface are `public abstract` by default** — you need not write either keyword, and a body is not allowed (*"Abstract methods do not specify a body"*).
- Because the interface method is public, the implementing method **must also be public**. Leaving it package-private fails with *"Cannot reduce the visibility of the inherited method from Car"*. The Java rule: an overriding method may widen access but never narrow it.

If a class does not want to define every method, it must itself be declared `abstract`; then a further concrete class must extend it and supply the missing definitions:

```
abstract class Thar implements Car { }          // does not define drive(), so it is abstract

class BlackThar extends Thar {
    @Override
    public void drive() { System.out.println("Black Thar is driving"); }
}
```



the first concrete class in the chain must define every method

The obligation to define every method passes down until a concrete class takes it.

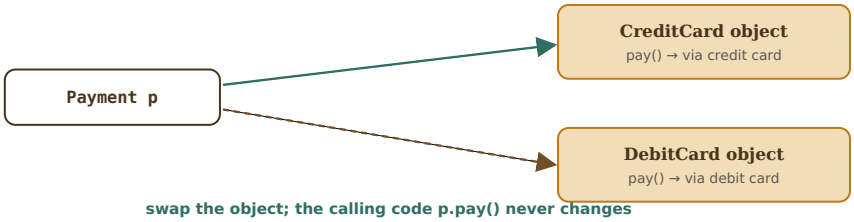
Dynamic polymorphism

An interface reference can point to any object whose class implements it, and the call runs that object's version of the method. That gives a big benefit — you can swap the implementation without touching the code that uses it:

```
interface Payment { void pay(); }

class CreditCard implements Payment {
    @Override public void pay() { System.out.println("Paying via credit card"); }
}
class DebitCard implements Payment {
    @Override public void pay() { System.out.println("Paying via debit card"); }
}

Payment p = new CreditCard();      // change this one line to new DebitCard() ...
p.pay();                           // Paying via credit card   (... and this call is untouched)
```



swap the object; the calling code p.pay() never changes

Which pay() runs is decided at run time by the object the reference holds.

> This idea underlies **dependency injection** in Spring Boot and clean architecture in general: code depends on an interface, and the actual implementation is chosen elsewhere. It will make fuller sense when Spring Boot is studied.

Variables in an Interface

An interface can hold variables too, but they are always constants: whatever you declare is automatically `public static final`. `static` because an interface has nothing to do with objects (you cannot make one), so it cannot hold per-object data; `final` because an interface may only define constants. That is why the names are written in capitals even though you did not write `final`.

```
interface MathConstant {
    double PI_VALUE = 3.14;          // really: public static final double PI_VALUE = 3.14;
    int VALUE = 10;
    void fun();                      // add this only if the interface should expose it
}

class Random implements MathConstant {
    @Override public void fun() { System.out.println(PI_VALUE); }   // readable, not changeable
}

System.out.println(MathConstant.PI_VALUE);      // 3.14  (no object, just the interface name)
```

- An implementing class can **read** the constant but not modify it. If you refer to the object through the interface type (`MathConstant r1 = new Random();`), only methods declared in the interface are visible — so `r1.fun()` compiles only if `fun()` is declared in `MathConstant`.
- In real projects an interface holding only constants is a handy place for values used across the program. A class could do it too, but then you must write `public static final` on every line yourself.

Active Recall — Part 3

Q1. What does an interface describe, and why is it called a contract?
A. What something can do, not how; implementing classes promise to define every method.

Q2. Why must an implementing method be public?
A. Interface methods are public, and overriding cannot reduce visibility.

Q3. What if a class does not define all the interface's methods?
A. It must be declared abstract, and a concrete subclass must define them.

Q4. What is the benefit of dynamic polymorphism with interfaces?
A. The implementation can be swapped without changing the calling code.

Q5. What modifiers does a variable in an interface get, and why?
A. public static final: no objects exist for an interface, and it may only hold constants.

Multiple Inheritance Through Interfaces

A class cannot extend two classes: `class C extends A, B` is a syntax error. But a class can **implement any number of interfaces**: `class C implements A, B` is fine, and C must then define the methods of both. A class still extends only one class.

```
interface A { void fun(); }
interface B { void fun2(); }

class C implements A, B {                       // two interfaces: allowed
    @Override public void fun()  { /* ... */ }
    @Override public void fun2() { /* ... */ }
}
// class C extends X, Y { }                     // two classes: syntax error
```

An interface **extends** another interface (never `implements` it — implementing means supplying definitions, and an interface only declares). It can add methods of its own, and an interface may extend several interfaces at once.

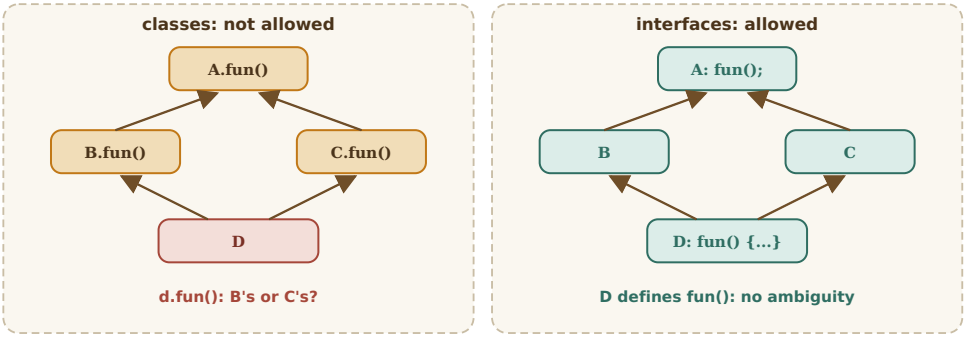
```
interface Animal { void eat(); }
interface Dog extends Animal { void bark(); }

class StreetDog implements Dog {                // must define eat() AND bark()
    @Override public void eat()  { System.out.println("Eating"); }
    @Override public void bark() { System.out.println("Barking"); }
}
```

The diamond problem

Why are classes not allowed multiple parents? Imagine class `A` with a fully defined `fun()`; classes `B` and `C` extend A and each overrides `fun()` differently; class `D` extends both B and C. For `d.fun()` should B's or C's version run? That ambiguity is the **diamond problem**. Java's designers (James Gosling's team) looked at the way other languages such as C++ solve it, judged the solution too complicated to be worth it, and simply banned multiple inheritance of classes.

With interfaces (before Java 8) the problem disappears: if A, B and C are interfaces they only *declare* `fun()` and no body exists to clash. The concrete class D that implements B and C has to define `fun()` itself, so there is exactly one answer.



Classes would leave D with two competing bodies; interfaces force D to supply the one definition.

Interface Changes After Java 8

Through Java 7 an interface could only declare methods. Later versions added three kinds of members that *have a body*:

Since	Member	Purpose
Java 8	`default` method	A method with a body; implementing classes inherit it and may override it, but need not.
Java 8	`static` method	Belongs to the interface; called as `InterfaceName.method()`, no object needed.
Java 9	`private` method	A helper usable only inside the interface itself (by its default and static methods).

```
interface Vehicle {
    default void drive() {                          // Java 8
        System.out.println("Vehicle is driving");
        accelerate();                               // calls the private helper
    }
    static void brake() {                           // Java 8
        System.out.println("Vehicle is applying brake");
    }
    private void accelerate() {                     // Java 9
        System.out.println("Vehicle is accelerating");
    }
}

class Car implements Vehicle { }                    // inherits drive() as is

Vehicle v = new Car();
v.drive();          // Vehicle is driving / Vehicle is accelerating
Vehicle.brake();    // Vehicle is applying brake  (called on the interface name)
// v.accelerate();  // compile error: private to the interface
```

If `Car` overrides `drive()` (written `public`), then `v.drive()` runs the Car version: the child class's own method takes priority over the interface's default.

Why default methods were introduced

The reason is **backward compatibility**. Java's `List` interface is implemented by countless classes in countless projects. If Java had added a new method to `List` as a plain abstract declaration, every one of those classes would suddenly fail to compile until it defined the new method — production code across the world would break. Declaring it as a `default` method with a body means existing classes keep working untouched, and any class that wants different behaviour can override it. (The same `default` idea means an interface with default methods now looks a lot like an abstract class with concrete methods.)

When two interfaces clash

Suppose `B` and `C` both give `fun()` a default body, and `D implements B, C`. Now the diamond ambiguity is back, so Java makes a rule: **D must override `fun()`**, otherwise it fails with *"Duplicate default methods named fun ... inherited from the types C and B"*. Inside the override D chooses what happens: its own code, one parent's version, or both — using `InterfaceName.super.method()`.

```
interface B { default void fun() { System.out.println("B"); } }
interface C { default void fun() { System.out.println("C"); } }

class D implements B, C {
    @Override
    public void fun() {            // mandatory: resolves the clash
        B.super.fun();             // run B's version ...
        C.super.fun();             // ... then C's  (or write your own code instead)
    }
}
new D().fun();                     // B  then  C
```

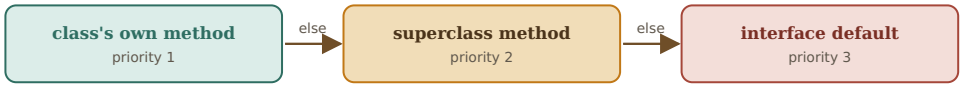
Class beats interface

A different clash has an automatic answer. If `C extends B implements A`, and both the **class** `B` and the **interface** `A` provide `fun()`, no error occurs — Java's **resolution priority rule** says the class's method always wins. (Override `fun()` in C itself if you want something else.)

```
interface A { default void fun() { System.out.println("Inside A interface"); } }
class B { public void fun() { System.out.println("Inside B class"); } }

class C extends B implements A { }       // no error, no override needed

new C().fun();                           // Inside B class   (the class wins)
```



two unrelated interface defaults clash → compile error → the class must override
inside the override: B.super.fun() or C.super.fun() or its own code

Which version of a method runs: the class itself, then its superclass, then interface defaults.

Active Recall — Part 4

Q1. How does Java allow multiple inheritance, and how does an interface inherit another?
A. A class may implement many interfaces; an interface uses extends (never implements) to inherit another.

Q2. What is the diamond problem and why does it vanish with plain interfaces?
A. Two parents give a child conflicting definitions; interfaces only declare, so the child supplies the one definition.

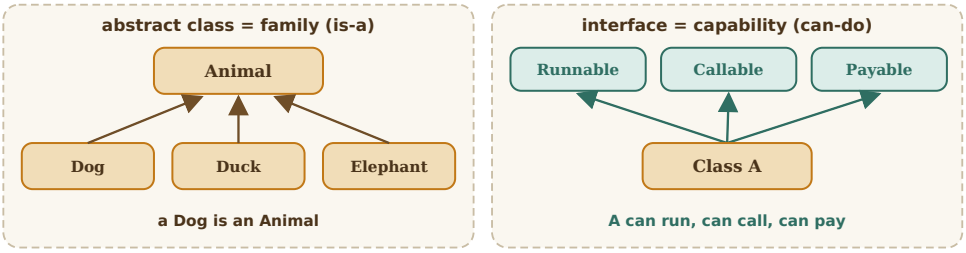
Q3. Name the three body-carrying interface members and their Java versions.
A. default (8), static (8), private (9).

Q4. Why were default methods introduced?
A. To add methods to existing interfaces (like List) without breaking every implementing class.

Q5. Two interfaces give the same default method — what must the class do? And class vs interface?
A. Override it, optionally calling B.super.fun() / C.super.fun(). When a class method and an interface default clash, the class method wins.

Interface vs Abstract Class

With default, static and private methods, an interface now looks a lot like an abstract class. They still differ, starting with **intent**. An interface describes **roles or capabilities** — what a class *can do* — which is why interfaces are often named like verbs or abilities (`Runnable` , `Callable` , `Payable`). An abstract class groups a **family of similar classes** under one parent so shared code can live once — an *is-a* relationship: a Dog is an Animal, so common behaviour such as `eat()` is defined once in `Animal` . An interface expresses a *can-do* relationship: a class that implements `Runnable` can run.



Abstract class: who you are (a family). Interface: what you can do (a role).

Aspect	Interface	Abstract class
Intent	Roles / capabilities (can-do)	Family of similar classes (is-a)
Instance variables	Not allowed — only `public static final` constants	Allowed, any kind of field
Constructors	None	Allowed
Multiple inheritance	A class can implement many	A class can extend only one
Method access	`public` by default (private allowed since Java 9)	Any access modifier

The reasoning behind the table: constructors belong to objects and an interface has nothing to do with objects, so it has none; likewise any non-static variable would be tied to an object, so only constants are possible.

Functional and Marker Interfaces

- **Functional interface** — an interface with exactly **one abstract method**. It unlocks **functional programming** through lambda expressions (a separate topic, introduced in Java 8). Java already ships many, such as `Comparable` and `Predicate` .
- **Marker interface** — an interface with **no methods at all**. It exists only to mark a class so that a capability is unlocked or checked. Standard examples: `Cloneable` , `Serializable` and `RandomAccess` .

```
interface Runner { void run(); }      // functional interface: exactly one abstract method
interface Tag { }                     // marker interface: no methods

class Student implements Cloneable {  // the marker lets Object.clone() proceed
    @Override
    protected Object clone() throws CloneNotSupportedException { return super.clone(); }
}
```

`Object` has a `clone()` method, but Java does not want everything to be cloneable (a database resource or a thread should not be duplicated). A class must therefore declare `implements Cloneable`; the interface has no methods to define — it simply tells Java "this class may be cloned". Without it, cloning fails at run time.

An interface is a class underneath

Like an enum, an interface is compiled into an ordinary `.class` file; Java only gave us a special keyword. The file carries a flag, `ACC_INTERFACE`, which tells the JVM that this is an interface and not a normal class. Everything else is treated like a class.

Active Recall — Part 5

Q1. What is the conceptual difference between an interface and an abstract class?

A. Interface = roles/capabilities (can-do); abstract class = family of similar classes (is-a).

Q2. Name three practical differences.

A. Interfaces have no instance variables, no constructors, and support multiple inheritance; abstract classes allow all of those restrictions' opposites (fields, constructors, single parent).

Q3. What is a functional interface?

A. An interface with exactly one abstract method; it enables lambda expressions and functional programming.

Q4. What is a marker interface? Name the standard ones.

A. An interface with no methods used only as a tag: Cloneable, Serializable, RandomAccess.

Q5. What does an interface compile into?

A. A class file carrying the ACC_INTERFACE flag.